

Ingineria Programării

Cursul 6 – 1 Aprilie 2025

adiftene@info.uaic.ro

Cuprins

- ▶ Din Cursurile trecute...
- ▶ SOLID Principles
- ▶ Design Patterns
 - Definitions
 - Elements
 - Example
 - Classification
- ▶ JUnit Testing
 - Netbeans (Exemplu 1)
 - Eclipse (Exemplu 2)

Din Cursurile Trecute

- ▶ Etapele Dezvoltării Programelor
- ▶ Ingineria Cerințelor
- ▶ Diagrame UML
- ▶ SOLID
- ▶ GRASP

R – GRASP

- ▶ Principii, responsabilități
- ▶ Information Expert
- ▶ Creator
- ▶ Low Coupling
- ▶ High Cohesion
- ▶ Controller

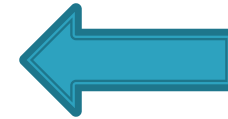
R – SOLID and Other Principles

- ▶ SOLID Principles
 - SRP – Single Responsibility Principle
 - OCP – Open/Closed Principle
 - LSP – Liskov Substitution Principle
 - ISP – Interface Segregation Principle
 - DIP – Dependency Inversion Principle
- ▶ DRY – Don't Repeat Yourself
- ▶ YAGNI – You Aren't Gonna Need It
- ▶ KISS – Keep It Simple, Stupid



Cuprins

- ▶ Din Cursurile trecute...
- ▶ Design Patterns
 - Definitions
 - Elements
 - Example
 - Classification
- ▶ JUnit Testing
 - Netbeans (Exemplu 1)
 - Eclipse (Exemplu 2)



Design Patterns – Why?

- ▶ If a problem occurs over and over again, a solution to that problem has been used effectively (solution = pattern)
- ▶ When you make a design, you should know the **names of some common solutions**. Learning design patterns is good for people to **communicate each other effectively**

Design Patterns – Definitions

- ▶ “Design patterns capture solutions that have developed and evolved over time” (GOF – ***Gang-Of-Four*** (because of the four authors who wrote it), *Design Patterns: Elements of Reusable Object-Oriented Software*)
- ▶ In software engineering (or computer science), a design pattern is a general repeatable solution to a commonly occurring problem in software design
- ▶ The **design patterns** are language-independent strategies for solving common object-oriented design problems

Gang of Four

- ▶ Initial was the name given to a leftist political faction composed of four Chinese Communist party officials
- ▶ The name of the book (“Design Patterns: Elements of Reusable Object–Oriented Software”) is too long for e–mail, so “book by the gang of four” became a shorthand name for it
- ▶ That got shortened to “**GOF book**“. Authors are: *Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides*
- ▶ The **design patterns** in their book are *descriptions of communicating objects and classes that are customized to solve a general design problem in a particular context*

Design Patterns – Elements

1. **Pattern name**
2. **Problem**
3. **Solution**
4. **Consequences**

Design Patterns – Pattern name

- ▶ A handle **used to describe a design problem**, its solutions, and consequences in a word or two
- ▶ Naming a pattern immediately increases our **design vocabulary**. It lets us design at a higher level of abstraction
- ▶ Having a **vocabulary** for patterns lets us talk about them with our colleagues, in our documentation
- ▶ Finding good names has been one of the **hardest parts of developing our catalog**

Design Patterns – Problem

- ▶ Describes **when** to apply the pattern. It explains the problem and its **context**
- ▶ It might describe specific design problems such as how to represent **algorithms** as objects
- ▶ It might describe **class** or **object** structures that are symptomatic of an inflexible design
- ▶ Sometimes the problem will include a **list of conditions** that must be met before it makes sense to apply the pattern

Design Patterns – Solution

- ▶ Describes the elements that make up the **design**, **their relationships**, **responsibilities**, and **collaborations**
- ▶ The solution **doesn't describe a particular concrete design or implementation**, because a pattern is like a template that can be applied in many different situations
- ▶ Instead, the pattern provides an **abstract description of a design problem** and how a general arrangement of elements (classes and objects in our case) **solves it**

Design Patterns – Consequences

- ▶ Are the results and trade-offs of applying the pattern
- ▶ They are critical for **evaluating design alternatives** and for **understanding the costs and benefits** of applying the pattern
- ▶ The consequences for software often concern **space and time trade-offs**, they can address **language and implementation issues** as well
- ▶ Include its impact on a system's **flexibility, extensibility, or portability**
- ▶ Listing these consequences explicitly helps you **understand and evaluate** them

Example of (Micro) pattern

- ▶ **Pattern name:** Initialization
- ▶ **Problem:** It is important for some code sequence to be executed only once at the beginning of the execution of the program.
- ▶ **Solution:** The solution is to use a static variable that holds information on whether or not the code sequence has been executed.
- ▶ **Consequences:** The solution requires the language to have a static variable that can be allocated storage at the beginning of the execution, initialized prior to the execution and remain allocated until the program termination.

Describing Design Patterns 1

- ▶ **Pattern Name and Classification**
- ▶ **Intent** – the answer to question: *What does the design pattern do?*
- ▶ **Also Known As**
- ▶ **Motivation** – A scenario that illustrates a design problem and how the class and object structures in the pattern solve the problem
- ▶ **Applicability** – *What are the situations in which the design pattern can be applied? How can you recognize these situations?*
- ▶ **Related Patterns**

Describing Design Patterns 2

- ▶ **Structure** – A graphical representation of the classes in the pattern
- ▶ **Participants** – The classes and/or objects participating in the design pattern and their responsibilities
- ▶ **Collaborations** – How the participants collaborate to carry out their responsibilities
- ▶ **Consequences** – *How does the pattern support its objectives?*
- ▶ **Implementation** – *What techniques should you be aware of when implementing the pattern?*
- ▶ **Sample Code**
- ▶ **Known Uses** – Examples of the pattern found in real systems

Design Patterns – Classification

- ▶ **Creational patterns**
- ▶ **Structural patterns**
- ▶ **Behavioral patterns**
- ▶ NOT in GOF: Fundamental, Partitioning, GRASP, GUI, Organizational Coding, Optimization Coding, Robustness Coding, Testing, Transactions, Distributed Architecture, Distributed Computing, Temporal, Database, Concurrency patterns

Creational Patterns

- ▶ **Abstract Factory** groups object factories that have a common theme
- ▶ **Builder** constructs complex objects by separating construction and representation
- ▶ **Factory Method** creates objects without specifying the exact class to create
- ▶ **Prototype** creates objects by cloning an existing object
- ▶ **Singleton** restricts object creation for a class to only one instance
- ▶ Not in GOF book: Lazy initialization, Object pool, Multiton, Resource acquisition (is initialization)

Structural Patterns

- ▶ **Adapter** allows classes with incompatible interfaces to work together
- ▶ **Bridge** decouples an abstraction from its implementation so that the two can vary independently
- ▶ **Composite** composes zero-or-more similar objects so that they can be manipulated as one object.
- ▶ **Decorator** dynamically adds/overrides behavior in an existing method of an object
- ▶ **Facade** provides a simplified interface to a large body of code
- ▶ **Flyweight** reduces the cost of creating and manipulating a large number of similar objects
- ▶ **Proxy** provides a placeholder for another object to control access, reduce cost, and reduce complexity

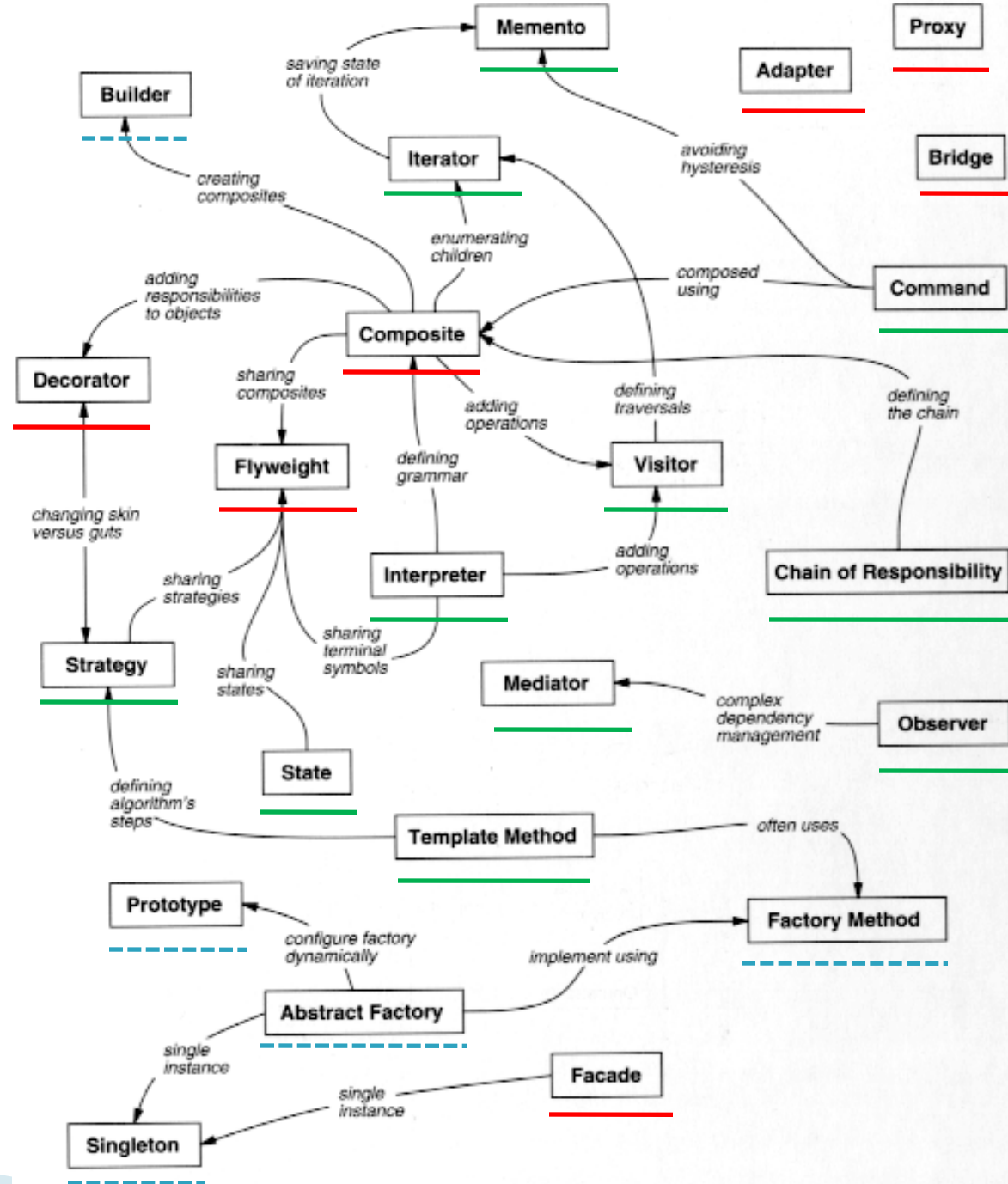
Behavioral patterns 1

- ▶ **Chain of responsibility** delegates commands to a chain of processing objects
- ▶ **Command** creates objects which encapsulate actions and parameters
- ▶ **Interpreter** implements a specialized language
- ▶ **Iterator** accesses the elements sequentially
- ▶ **Mediator** allows loose coupling between classes by being the only class that has detailed knowledge of their methods
- ▶ **Memento** provides the ability to restore an object to its previous state

Behavioral patterns 2

- ▶ **Observer** allows to observer objects to see an event
- ▶ **State** allows an object to alter its behavior when its internal state changes
- ▶ **Strategy** allows one of a family of algorithms to be selected on-the-fly at runtime
- ▶ **Template** defines an algorithm as an abstract class, allowing its subclasses to provide concrete behavior
- ▶ **Visitor** separates an algorithm from an object structure
- ▶ Not in GOF book: Null Object, Specification

- Patterns
 - Creational
 - Structural
 - Behavioral



How to Select a Design Pattern?

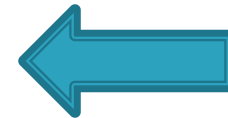
- ▶ With more than 20 design patterns to choose from, it might be hard to find the one that addresses a particular design problem
- ▶ Approaches to finding the design pattern that's right for your problem:
 1. *Consider how design patterns solve design problems*
 2. *Scan Intent sections*
 3. *Study relationships between patterns*
 4. *Study patterns of like purpose (comparison)*
 5. *Examine a cause of redesign*
 6. *Consider what should be variable in your design*

How to Use a Design Pattern?

1. *Read the pattern once through for an overview*
2. *Go back and study the Structure, Participants, and Collaborations sections*
3. *Look at the Sample Code section to see a concrete example*
4. *Choose names for pattern participants that are meaningful in the application context*
5. *Define the classes*
6. *Define application-specific names for operations in the pattern*
7. *Implement the operations to carry out the responsibilities and collaborations in the pattern*

Cuprins

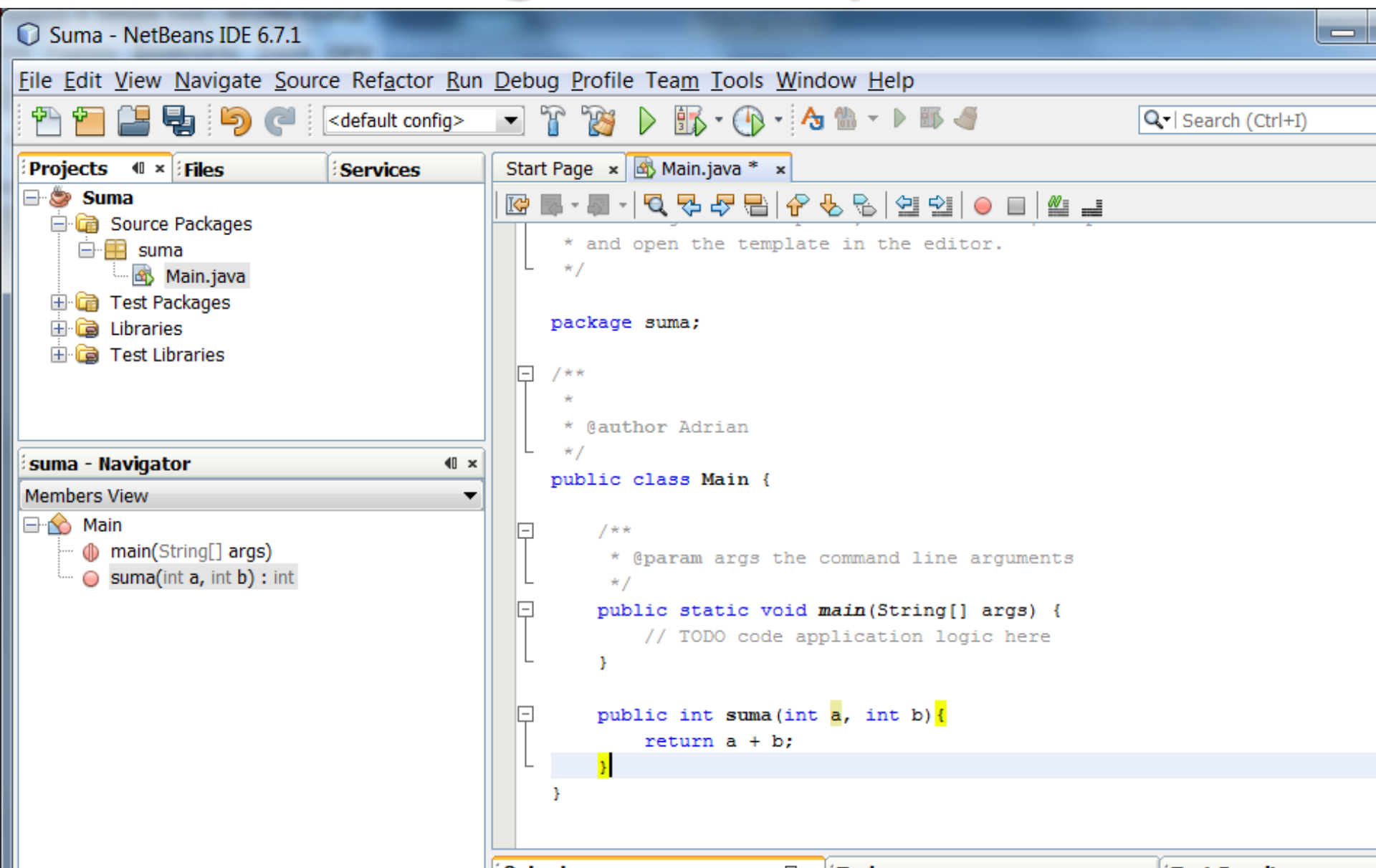
- ▶ Din Cursurile trecute...
- ▶ Design Patterns
 - Definitions
 - Elements
 - Example
 - Classification
- ▶ JUnit Testing
 - Netbeans (Exemplu 1)
 - Eclipse (Exemplu 2)



Unit Testing

- ▶ Testarea unei funcții, a unui program, a unui ecran, a unei funcționalități
- ▶ Se face de către programatori
- ▶ Predefinită
- ▶ Rezultatele trebuie documentate
- ▶ Se folosesc simulatoare pentru Input și Output

Unit Testing – Exemplan 1 (1)



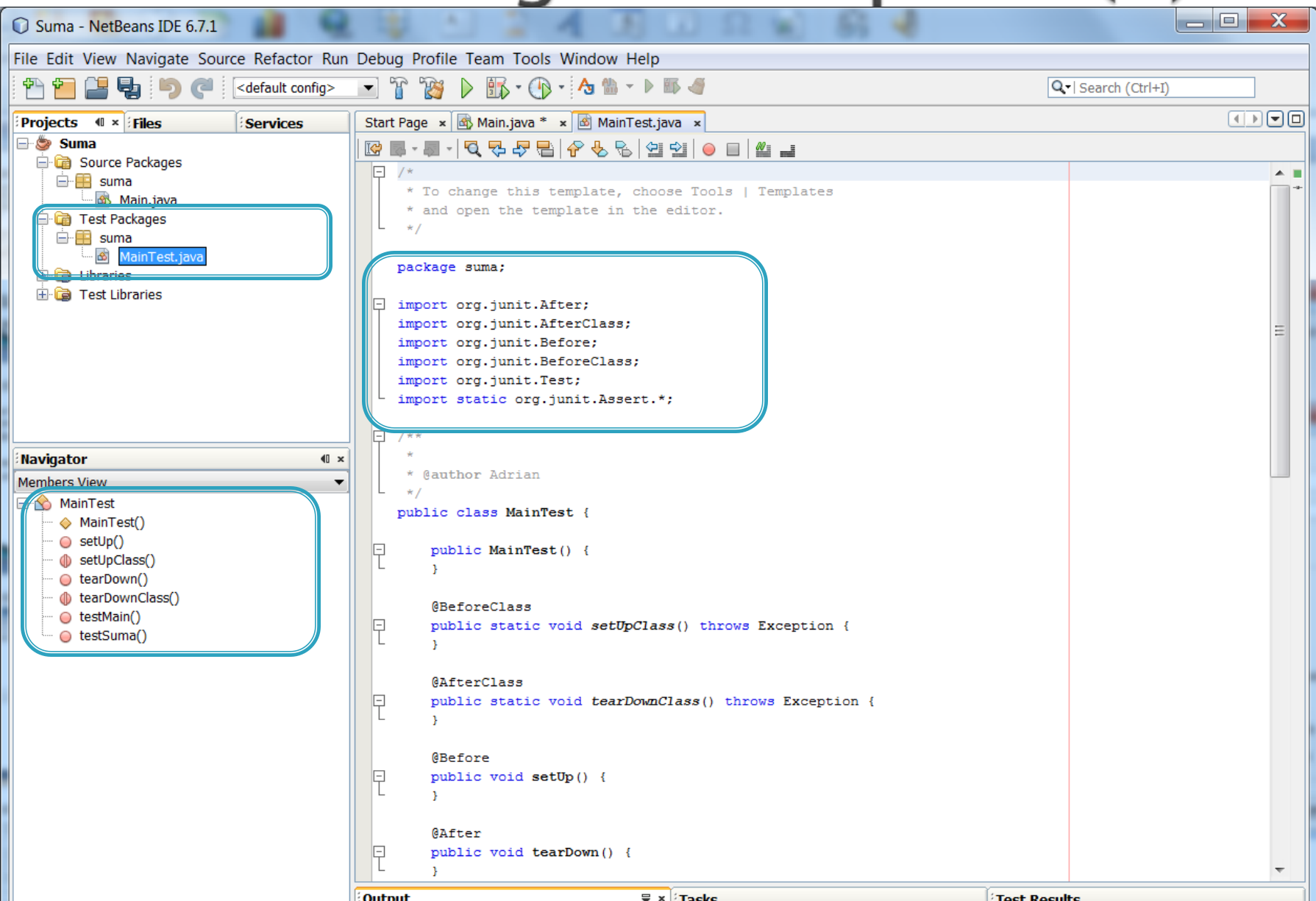
Unit Testing – Exemplan 1 (2)

The screenshot displays an IDE interface with the following components:

- Projects Panel:** Shows a project named 'Suma' with sub-packages 'Source Packages', 'suma', 'Test Packages', 'Libraries', and 'Test Libraries'. The 'Main.java' file is selected under 'suma'.
- suma - Navigator:** Shows the 'Main' class with methods 'main(String[] a)' and 'suma(int a, int b)'.
- Main.java Editor:** Contains the following code:

```
/* To change this template, choose Tools | Templates  
 * To open the template in the editor.  
 */  
  
package suma;  
  
import java.io.IOException;  
  
/**  
 *  
 * @author Adrian  
 */  
class Main {  
  
    /**  
     * param args the command line arguments  
     */  
    public static void main(String[] args) {  
        // TODO code application logic here  
    }  
  
    public int suma(int a, int b) {  
        return a + b;  
    }  
}
```
- Context Menu:** A right-click menu is open over the 'suma(int a, int b)' method, showing options like 'Open', 'Cut', 'Copy', 'Paste', 'Compile File', 'Run File', 'Debug File', 'Profile File', 'Test File', 'Add', 'Delete', 'Save As Template...', 'Find Usages', 'Refactor', 'BeanInfo Editor...', 'File Members', 'File Hierarchy', 'Local History', 'Tools', and 'Properties'. The 'Tools' menu is expanded, showing 'Add to Favorites', 'Create JUnit Tests' (highlighted), 'Analyze Javadoc', and 'Add to Palette...'. The 'Create JUnit Tests' option has the shortcut 'Ctrl+Shift+T'.
- Select JUnit Version Dialog:** A dialog box titled 'Select JUnit Version' with the text 'Select JUnit version for which the created test skeletons should be created:'. It has two radio buttons: 'JUnit 3.x' (selected) and 'JUnit 4.x'.
- Create Tests Dialog:** A dialog box titled 'Create Tests' with the following fields and options:
 - Class to Test:** suma.Main
 - Class Name:** suma.MainTest
 - Location:** Test Packages
 - Code Generation:**
 - Method Access Levels:** ☒ Public, ☒ Protected, ☒ Package Private
 - Generated Code:** ☒ Test Initializer, ☒ Test Finalizer, ☒ Default Method Bodies
 - Generated Comments:** ☒ Javadoc Comments, ☒ Source Code Hints

Unit Testing – Exemplu 1 (3)



Unit Testing – Exemplan 1 (4)

IDE 6.7.1

File Source Refactor Run Debug Profile Team Tools Window Help

<default config>

Services

Start Page x

Main.java * x

MainTest.java x

/**

* Test of suma method, of class Main.

*/

@Test

public void testSuma() {

System.out.println("suma");

int a = 0;

int b = 0;

Main instance = new Main();

int expectedResult = 0;

int result = instance.suma(a, b);

assertEquals(expectedResult, result);

// TODO review the generated test code and remove the default call

fail("The test case is a prototype.");

}

}

Unit Testing – Exemplu 1 (5)

The screenshot displays an IDE interface for unit testing. The **Projects** panel on the left shows the project structure with **MainTest.java** selected. A context menu is open over **MainTest.java**, with **Run File** (Shift+F6) highlighted. The **Navigator** panel shows the **MainTest** class with methods **setUp()**, **tearDown()**, **testMain()**, and **testSuma()**.

The **MainTest.java** file is open, showing the following code:

```
@Before
public void setUp() {
}

@After
public void tearDown() {
}

/**
 * Test of main method, of class Main.
 */
@Test
public void testMain() {
    System.out.println("main");
    String[] args = null;
    Main.main(args);
    // TODO review the generated test code and remove the default call to fail.
    fail("The test case is a prototype.");
}

/**
 * Test of suma method, of class Main.
 */
@Test
public void testSuma() {
}
```

The **Output - Suma (test)** window shows the test results:

```
0.0 %
No test passed, 2 tests failed.(0.145 s)
suma.MainTest FAILED
  testMain FAILED (at suma.MainTest.testMain(MainTest.java:49))
    at suma.MainTest.testMain(MainTest.java:49)
  testSuma FAILED (at suma.MainTest.testSuma(MainTest.java:65))
    at suma.MainTest.testSuma(MainTest.java:65)
```

The **Test Results** table shows the following results:

Test Name	Result
main	Failed
suma	Failed

Unit Testing – Exemplan 1 (6)

```
@Test
public void testSuma() {
    System.out.println("suma");
    int a = 0;
    int b = 0;
    Main instance = new Main();
    int expectedResult = 0;
    int result = instance.suma(a, b);
    assertEquals(expResult, result);
    // TODO review the generated test code and remove the default call to fail.
    //fail("The test case is a prototype.");
}
```

Output - Suma (test)

Tasks

Test Results



50.0 %

1 test passed, 1 test failed.(0.019 s)

suma.MainTest FAILED

testMain FAILED (at suma.MainTest.testMain(MainTest.java:49))

testSuma passed (0.0 s)

main
suma

Unit Testing – Example 2 (1)

BasicOperations.java BasicOperationsTest.java

```
package math;

public class BasicOperations {

    public int add(int x, int y) {
        return x + y;
    }

    public int min(int x, int y) {
        return x + y;
    }

    public int mul(int x, int y) {
        return x * y;
    }

    public int div(int x, int y) {
        return x / y;
    }

    /**
     * @param args
     */
    public static void main(String[] args) {
        // TODO Auto-generated method stub
        BasicOperations bc = new BasicOperations();
        System.out.println(bc.add(3,5));
    }
}
```

Unit Testing – Example 2 (2)

Package Explorer | Hierarchy | JUnit

Finished after 0.019 seconds

Runs: 4/4 Errors: 0 Failures: 1

test.BasicOperationsTest [Runner: JUnit 4] (0.004 s)

- testAdd (0.000 s)
- testMin (0.003 s)
- testMul (0.000 s)
- testDiv (0.001 s)

Failure Trace

java.lang.AssertionError: Result expected:<2> but was:<8>
at test.BasicOperationsTest.testMin(BasicOperationsTest.java:20)

BasicOperations.java | BasicOperationsTest.java

```
package test;

import static org.junit.Assert.*;

public class BasicOperationsTest {

    @Test
    public void testAdd() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 8, bo.add(3, 5));
    }

    @Test
    public void testMin() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 2, bo.min(5, 3));
    }

    @Test
    public void testMul() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 15, bo.mul(3, 5));
    }

    @Test
    public void testDiv() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 2, bo.div(4, 2));
    }
}
```

Unit Testing – Example 2 (3)

The screenshot shows an IDE with two main panels. The left panel displays the results of a JUnit test run. The top section shows 'Finished after 0.019 seconds' and a summary: 'Runs: 4/4', 'Errors: 1', and 'Failures: 0'. Below this, a list of test methods is shown: 'testAdd (0.000 s)', 'testMin (0.000 s)', 'testMul (0.000 s)', and 'testDiv (0.004 s)'. The 'testDiv' method is highlighted with a red 'x' icon, indicating a failure. The bottom section, titled 'Failure Trace', shows the exception: 'java.lang.ArithmeticException: / by zero', with the stack trace pointing to 'math.BasicOperations.div(BasicOperations.java:18)' and 'test.BasicOperationsTest.testDiv(BasicOperationsTest.java:32)'. The right panel shows the source code for 'BasicOperationsTest.java'. The code defines a 'test' package, imports 'org.junit.Assert.*', and contains a 'BasicOperationsTest' class with four test methods: 'testAdd()', 'testMin()', 'testMul()', and 'testDiv()'. The 'testDiv()' method is highlighted with a blue box, showing it calls 'bo.div(4, 0)' and asserts the result is 0. The 'BasicOperations.java' file is also visible in the background.

Package Explorer | Hierarchy | JUnit

Finished after 0.019 seconds

Runs: 4/4 Errors: 1 Failures: 0

test.BasicOperationsTest [Runner: JUnit 4] (0.004 s)

- testAdd (0.000 s)
- testMin (0.000 s)
- testMul (0.000 s)
- testDiv (0.004 s)

Failure Trace

java.lang.ArithmeticException: / by zero

at math.BasicOperations.div(BasicOperations.java:18)

at test.BasicOperationsTest.testDiv(BasicOperationsTest.java:32)

BasicOperations.java | BasicOperationsTest.java

```
package test;

import static org.junit.Assert.*;

public class BasicOperationsTest {

    @Test
    public void testAdd() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 8, bo.add(3, 5));
    }

    @Test
    public void testMin() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 2, bo.min(5, 3));
    }

    @Test
    public void testMul() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 15, bo.mul(3, 5));
    }

    @Test
    public void testDiv() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 0, bo.div(4, 0));
    }
}
```

Unit Testing – Example 2 (4)

Package Explorer | Hierarchy | JUnit

Finished after 0.016 seconds

Runs: 4/4 Errors: 0 Failures: 0

- test.BasicOperationsTest [Runner: JUnit 4] (0.000 s)
 - testAdd (0.000 s)
 - testMin (0.000 s)
 - testMul (0.000 s)
 - testDiv (0.000 s)

Failure Trace

BasicOperations.java | BasicOperationsTest.java

```
package test;

import static org.junit.Assert.*;

public class BasicOperationsTest {

    @Test
    public void testAdd() {
        BasicOperations bo = new BasicOperations();
        assertTrue("Result", 8 == bo.add(3, 5));
    }

    @Test
    public void testMin() {
        BasicOperations bo = new BasicOperations();
        assertFalse("Result", ! (3 != bo.min(5, 3)));
    }

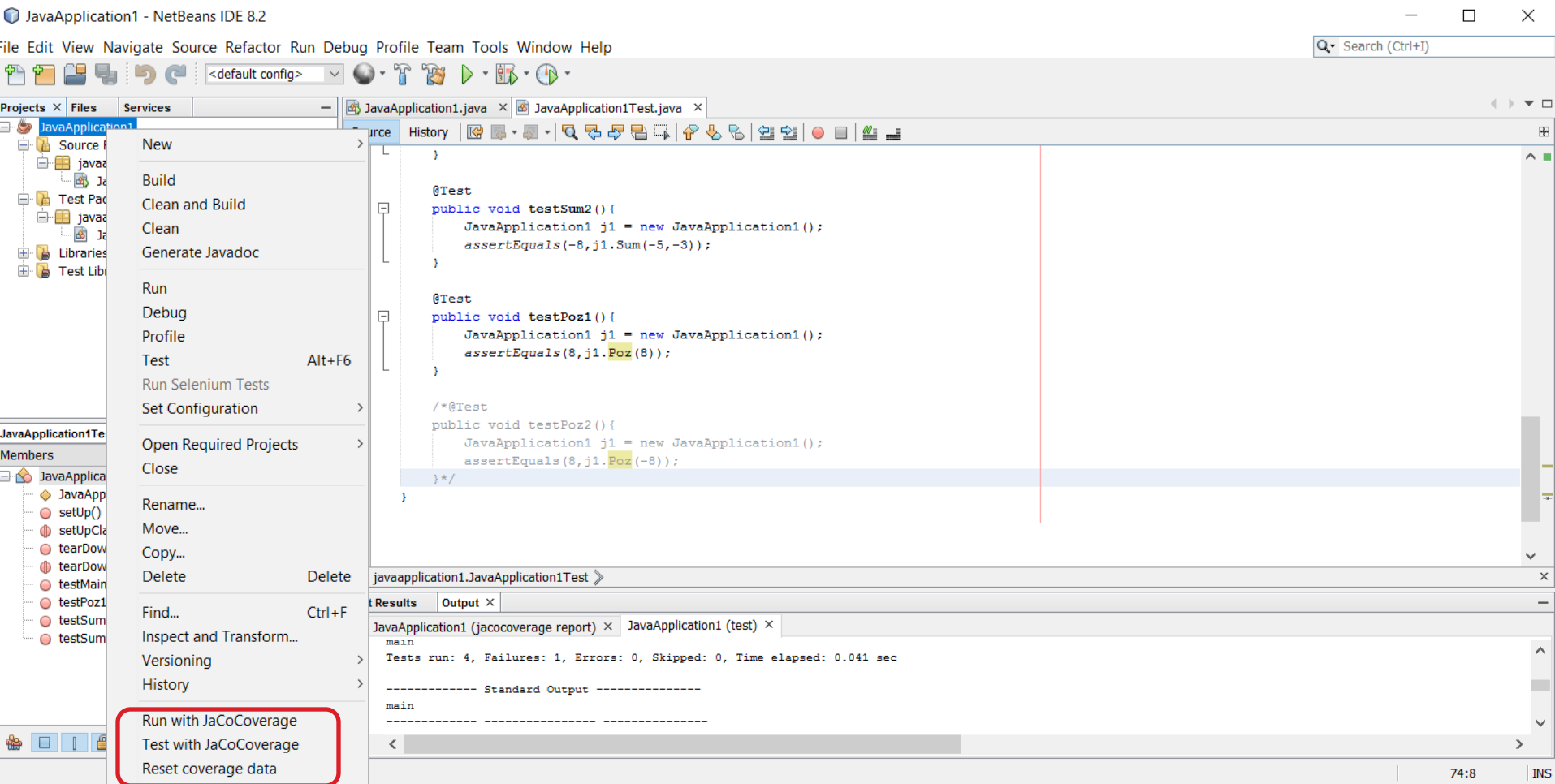
    @Test
    public void testMul() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 15, bo.mul(3, 5));
    }

    @Test
    public void testDiv() {
        BasicOperations bo = new BasicOperations();
        if(bo.div(4, 2) == 3)
            fail("Incorrect result!");
    }
}
```

Code Coverage

- ▶ NetBeans – TikiOne JaCoCoverage:
 - ▶ <http://plugins.netbeans.org/plugin/48570/tikione-jacocoverage>
- ▶ Java Code Coverage for Eclipse:
 - ▶ <http://www.eclemma.org/>
- ▶ IntelliJ – Running with coverage:
 - ▶ <https://www.jetbrains.com/help/idea/2016.3/running-with-coverage.html>

NetBeans – 1



NetBeans – 2

JavaApplication1 - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

Search (Ctrl+I)

Projects Files Services

JavaApplication1

Source History

@Test

public void testSum2()

Adrian

JaCoCoverage analysis of project "JavaApplication1" (powered by JaCoCo from EcJemma)

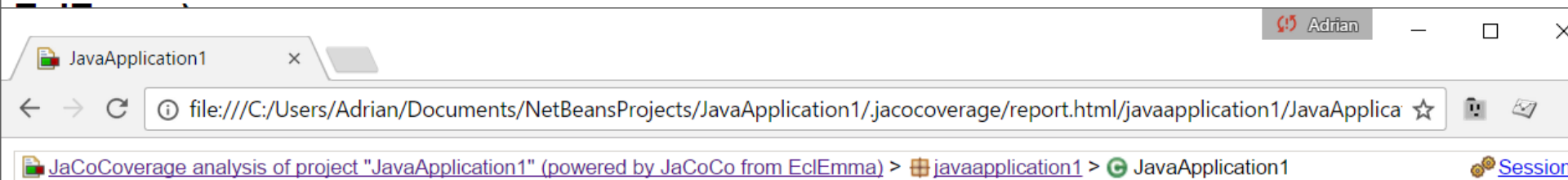
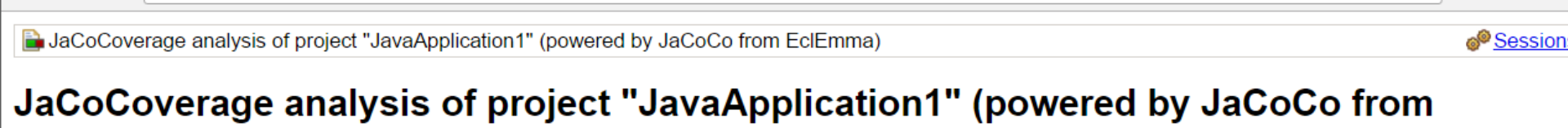
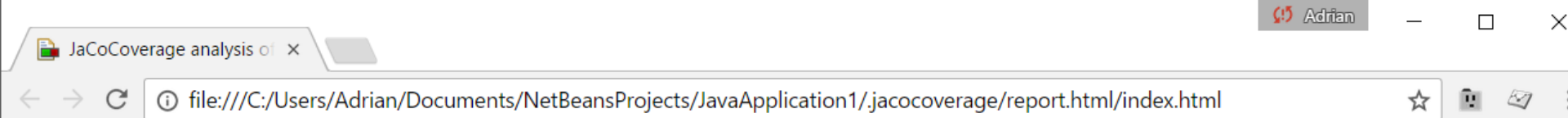
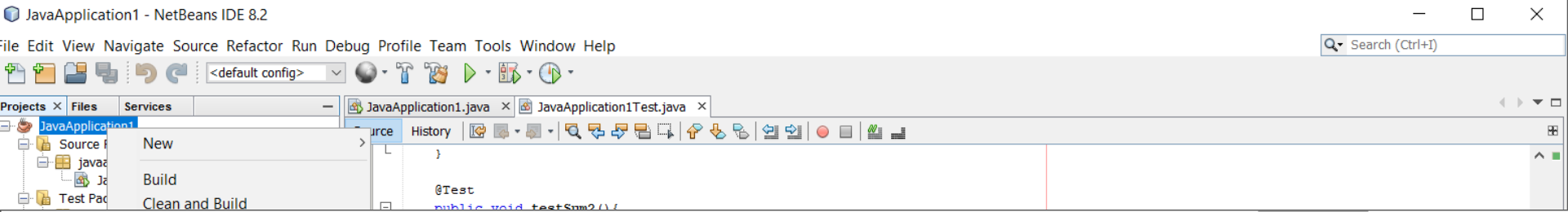
file:///C:/Users/Adrian/Documents/NetBeansProjects/JavaApplication1/jacocoverage/report.html/index.html

JaCoCoverage analysis of project "JavaApplication1" (powered by JaCoCo from EcJemma)

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
javaapplication1		80%		50%	1	5	1	7	0	4	0	1
Total	4 of 20	80%	1 of 2	50%	1	5	1	7	0	4	0	1

Created with [JaCoCo](#) 0.7.6.2016021808

NetBeans – 3



JavaApplication1										
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
Poz(int)		50%		50%	1	2	1	3	0	1
main(String[])		100%		n/a	0	1	0	2	0	1
Sum(int, int)		100%		n/a	0	1	0	1	0	1
JavaApplication1()		100%		n/a	0	1	0	1	0	1
Total	4 of 20	80%	1 of 2	50%	1	5	1	7	0	4

Using AI to Generate Testing Code

- ▶ Using AI Coding assistant (GitHub Copilot, Microsoft Copilot, ChatGPT)
- ▶ Prompt based Testing Code Generation (Claude)
- ▶ Custom AI models (fine tuned GPT models)
- ▶ *create a Java class for student with a method that verify the year of a student*
- ▶ *Develop a comprehensive suite of unit tests for the student. Write multiple test methods that cover a wide range of scenarios, including edge cases, exception handling, and data validation.*

<https://github.com/copilot/>

github.com/copilot/c/666f92ab-41d4-4b2d-a34a-b7bdbb4e8e30

Q Type to search

+

⌂

🔗

📧

👤

⋮

☰

🐙 Copilot

📄

📝

GPT-4o

🔗

⋮

create a Java class for student with a method that verify the year of a student

👤

Here's a Java class for a `Student` with a method to verify the year of the student:

<> Student.java v3

```
public class Student {
    private String name;
    private int year;

    public Student(String name, int year) {
```

This `Student` class has:

- Two private fields: `name` and `year`.
- A constructor to initialize these fields.
- Getter methods for `name` and `year`.
- A `verifyYear` method that checks if the student's year matches the expected year.
- A `main` method to demonstrate the use of the `verifyYear` method.

Feel free to add more methods or modify it as needed!

Ask Copilot

🔗

📝

➤

📄

<> Person.java

<> Professor.java

<> Student.java x

<> StudentTest

⋮

v3

26 lines · 602 bytes

📄

📥

```
1 public class Student {
2     private String name;
3     private int year;
4
5     public Student(String name, int year) {
6         this.name = name;
7         this.year = year;
8     }
9
10    public String getName() {
11        return name;
12    }
13
14    public int getYear() {
15        return year;
16    }
17
18    public boolean verifyYear(int expectedYear) {
19        return this.year == expectedYear;
20    }
21
22    public static void main(String[] args) {
23        Student student = new Student("John Doe", 2023);
24        System.out.println("Student Year Verification: " + student.verifyYear(2023));
25    }
26 }
```

Use `Control + Shift + m` to toggle the `tab` key moving focus. Alternatively, use `esc` then `tab` to move to the next interactive element on the page.

Q Type to search

+ -

<> Person.java

<> Professor.java

<> Student.java

<> StudentTes ...

v3 69 lines · 2 KB

```
1 import org.junit.jupiter.api.Test;
2 import static org.junit.jupiter.api.Assertions.*;
3
4 public class StudentTest {
5
6     @Test
7     public void testVerifyYear_CorrectYear() {
8         Student student = new Student("John Doe", 2023);
9         assertTrue(student.verifyYear(2023), "The year should be 2023");
10    }
11
12    @Test
13    public void testVerifyYear_IncorrectYear() {
14        Student student = new Student("John Doe", 2023);
15        assertFalse(student.verifyYear(2022), "The year should not be 2022");
16    }
17
18    @Test
19    public void testVerifyYear_EdgeCaseYear() {
20        Student student = new Student("Jane Doe", 0);
21        assertTrue(student.verifyYear(0), "The year should be 0");
22    }
23
24    @Test
25    public void testVerifyYear_NegativeYear() {
26        Student student = new Student("Jane Doe", -1);
```

Use **Control + Shift + m** to toggle the **tab** key moving focus. Alternatively, use **esc** then **tab** to move to the next interactive element on the page.

Q Type to search

+ -

<> Person.java

<> Professor.java

<> Student.java

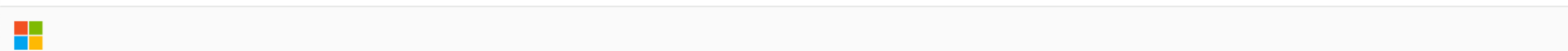
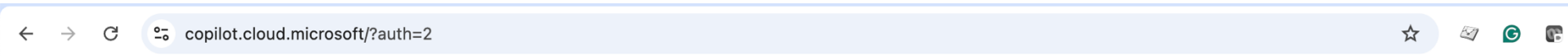
<> StudentTes ...

v3 69 lines · 2 KB

```
44 @Test
45 public void testGetName() {
46     Student student = new Student("John Doe", 2023);
47     assertEquals("John Doe", student.getName(), "Name should be John Doe");
48 }
49
50 @Test
51 public void testGetYear() {
52     Student student = new Student("John Doe", 2023);
53     assertEquals(2023, student.getYear(), "Year should be 2023");
54 }
55
56 @Test
57 public void testSetName() {
58     Student student = new Student("John Doe", 2023);
59     student.setName("Jane Doe");
60     assertEquals("Jane Doe", student.getName(), "Name should be Jane Doe");
61 }
62
63 @Test
64 public void testSetYear() {
65     Student student = new Student("John Doe", 2023);
66     student.setYear(2024);
67     assertEquals(2024, student.getYear(), "Year should be 2024");
68 }
69 }
```

Use **Control + Shift + m** to toggle the **tab** key moving focus. Alternatively, use **esc** then **tab** to move to the next interactive element on the page.

<https://copilot.cloud.microsoft/>



Copilot

create a Java class for student with a method that...

New chat



create a Java class for student with a method that verify the year of a student



Copilot

Sure! Here's a simple Java class for a student with a method to verify the year of the student:

```
public class Student {  
    private String name;  
    private int year;  
  
    // Constructor  
    public Student(String name, int year) {  
        this.name = name;  
        this.year = year;  
    }  
  
    // Getter for name  
    public String getName() {  
        return name;  
    }  
  
    // Getter for year  
    public int getYear() {  
        return year;  
    }  
}
```

Can you explain the test cases?

How to run these tests in an IDE?

What if I want to use Mockito?

View prompts

Ask me anything...

0 / 8000



Copilot

Agents

Get agents

Create an agent

Recent chats

create a Java class

Generate classes

[See more](#)



Copilot

create a Java class for student with a method that...

+ New chat



```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class StudentTest {

    @Test
    public void testConstructorAndGetters() {
        Student student = new Student("John Doe", 2025);
        assertEquals("John Doe", student.getName());
        assertEquals(2025, student.getYear());
    }

    @Test
    public void testVerifyYear() {
        Student student = new Student("John Doe", 2025);
        assertTrue(student.verifyYear(2025));
        assertFalse(student.verifyYear(2024));
    }

    @Test
    public void testVerifyYearWithEdgeCases() {
        Student student = new Student("John Doe", Integer.MAX_VALUE);
        assertTrue(student.verifyYear(Integer.MAX_VALUE));
        assertFalse(student.verifyYear(Integer.MIN_VALUE));
    }
}
```

Can you explain the test cases?

How to run these tests in an IDE?

What if I want to use Mockito?

🔖 View prompts

Ask me anything...

0 / 8000

Concluzii

- ▶ SOLID
- ▶ Design Patterns
 - Definitions, Elements, Example, Classification
- ▶ JUnit Testing

Myths

▶ Clients

- A general description of the objectives is sufficient to begin writing program
- Requirements are constantly changing, but the software is flexible and can easy adapts

▶ Developers

- Once the program is written and it is functional, our role has ended
- Until the program doesn't work, we can not assess the quality
- The only good product is the functional program
- Software Engineering will create voluminous and unnecessary documentation and will cause delays

Design Patterns – Întrebări

- ▶ 1) Argumentați pentru folosirea DP.
- ▶ 2) Veniți cu argumente pentru a nu folosi DP.
- ▶ Criticism:
http://sourcemaking.com/design_patterns

Bibliografie

- ▶ Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides: *Design Patterns: Elements of Reusable Object-Oriented Software* (GangOfFour)
- ▶ Ovidiu Gheorghieș, Curs 7 IP
- ▶ Adrian Iftene, Curs 9 TAIP:
<http://thor.info.uaic.ro/~adiftene/Scoala/2011/TAIP/Courses/TAIP09.pdf>

Links

- ▶ Gang-Of-Four: <http://c2.com/cgi/wiki?GangOfFour>,
<http://www.uml.org.cn/c%2B%2B/pdf/DesignPatterns.pdf>
- ▶ Design Patterns Book: <http://c2.com/cgi/wiki?DesignPatternsBook>
- ▶ About Design Patterns: <http://www.javacamp.org/designPattern/>
- ▶ Design Patterns – Java companion:
<http://www.patterndepot.com/put/8/JavaPatterns.htm>
- ▶ Java Design patterns:
http://www.allapplabs.com/java_design_patterns/java_design_patterns.htm
- ▶ Overview of Design Patterns:
http://www.mindspring.com/~mgrand/pattern_synopses.htm
- ▶ Gang of Four: http://en.wikipedia.org/wiki/Gang_of_four
- ▶ JUnit in Eclipse: <http://www.vogella.de/articles/JUnit/article.html>
- ▶ JUnit in NetBeans: <http://netbeans.org/kb/docs/java/junit-intro.html>

Vă Mulțumesc!

Pentru prezență,
răbdare,
colaborare...

